

Xây dựng hệ thống Agent điều phối (Orchestrator) và Sub-agent chuyên biệt

Tổng hợp toàn diện: Agent → Multi-agent → Context Engineering → Loop Engineering

Tổng hợp tài liệu tham khảo (2025-2026)

Tháng 7, 2026

Contents

Mục lục nội dung	2
1. Đặt vấn đề và kiến trúc mục tiêu	3
2. Giai đoạn 1 — Nền tảng về Agent	4
3. Giai đoạn 2 — Kiến trúc Multi-agent (Orchestrator-Worker)	6
4. Giai đoạn 3 — Context Engineering	11
5. Giai đoạn 4 — Loop Engineering	15
6. Tài liệu bổ trợ: quản trị agent, containment, human-agent teams	17
7. Tổng hợp nguyên tắc thiết kế then chốt	19
8. Áp dụng cụ thể vào bài toán trợ lý pháp lý đa lĩnh vực	20
9. Danh mục toàn bộ nguồn tham khảo	21

Mục lục nội dung

1. Đặt vấn đề và kiến trúc mục tiêu
2. Giai đoạn 1 — Nền tảng về Agent
3. Giai đoạn 2 — Kiến trúc Multi-agent (Orchestrator-Worker)
4. Giai đoạn 3 — Context Engineering
5. Giai đoạn 4 — Loop Engineering
6. Tài liệu bổ trợ: quản trị agent, containment, human-agent teams
7. Tổng hợp nguyên tắc thiết kế then chốt
8. Áp dụng cụ thể vào bài toán trợ lý pháp lý đa lĩnh vực
9. Danh mục toàn bộ nguồn tham khảo

1. Đặt vấn đề và kiến trúc mục tiêu

Mục tiêu là xây dựng một hệ thống trợ lý pháp lý gồm:

- Một **agent điều phối (orchestrator)** dùng model nhỏ, chi phí thấp (kiểu “flash-lite”), có nhiệm vụ chính là phân phối câu hỏi/tác vụ tới đúng nơi xử lý, hạn chế tối đa việc tự nó xử lý các tác vụ nặng.
- Orchestrator gọi xuống các **sub-agent chuyên biệt theo lĩnh vực/bộ luật** (ví dụ: agent luật đất đai, agent luật thuế...). Mỗi sub-agent có: (a) instruction rõ ràng theo định dạng chuẩn, (b) tool riêng để lấy dữ liệu/tìm kiếm, (c) skill gắn với một loại vấn đề cụ thể — mỗi skill biết cần dùng tool nào.
- Một tầng **quản lý ngữ cảnh (context management)**: giữ context, các thiết lập, lưu trí nhớ cá nhân (personal memory) để agent trả lời phù hợp hơn — hướng tới cơ chế “adaptive/deep thinking”.
- Một tầng sâu hơn: agent biết **lập kế hoạch (planning)** — có cơ chế suy nghĩ (thinking) để phân tích câu hỏi, tạo checklist, rồi giao việc; ví dụ luồng “nghiên cứu một bộ luật → xuất báo cáo PDF, hoặc gửi email” thông qua các hành động MCP (Model Context Protocol) cụ thể.
- Cách tiếp cận: bắt đầu từ cái cơ bản (một domain-agent xử lý đúng phạm vi của nó, ví dụ hỏi về thuế → trả lời đúng phạm vi bộ luật thuế), rồi mới nâng cấp dần lên kiến trúc multi-agent đầy đủ.

Bốn tài liệu nền tảng cần đọc theo đúng thứ tự: **Agent → Multi-agent → Context Engineering → Loop Engineering**. Đây cũng chính là trình tự phát triển thực tế của lĩnh vực này từ giữa năm 2025 đến giữa năm 2026.

2. Giai đoạn 1 — Nền tảng về Agent

2.1. “Building Effective Agents” (Anthropic — Erik Schluntz & Barry Zhang)

Nguồn: <https://www.anthropic.com/research/building-effective-agents>

Đây là tài liệu nền tảng, định nghĩa lại toàn bộ cách nhìn về “agent” trong ngành. Có nhiều định nghĩa khác nhau về agent: một số coi agent là hệ thống hoàn toàn tự động, hoạt động độc lập trong thời gian dài, dùng nhiều tool để hoàn thành các tác vụ phức tạp; số khác dùng thuật ngữ này để mô tả các cách triển khai mang tính quy định hơn, đi theo các workflow định sẵn.

Anthropic gộp tất cả các biến thể này vào khái niệm chung “hệ thống agentic” (agentic systems), nhưng vẫn phân biệt kiến trúc quan trọng giữa hai khái niệm:

- **Workflow:** hệ thống mà LLM và tool được điều phối thông qua các đường code định sẵn (predefined code paths).
- **Agent:** hệ thống mà LLM tự định hướng quy trình và cách sử dụng tool của chính nó, duy trì quyền kiểm soát về cách hoàn thành nhiệm vụ.

Nguyên tắc thiết kế cốt lõi: khi xây dựng ứng dụng với LLM, nên tìm giải pháp đơn giản nhất có thể, và chỉ tăng độ phức tạp khi thực sự cần thiết — điều này có thể có nghĩa là **không xây dựng hệ thống agentic nào cả**. Hệ thống agentic thường đánh đổi độ trễ (latency) và chi phí để lấy hiệu năng tác vụ tốt hơn, và cần cân nhắc kỹ khi nào sự đánh đổi này là hợp lý. Khi độ phức tạp cao hơn là cần thiết: workflow mang lại khả năng dự đoán và tính nhất quán cho các tác vụ được định nghĩa rõ ràng, trong khi agent là lựa chọn tốt hơn khi cần sự linh hoạt và ra quyết định do model dẫn dắt ở quy mô lớn. Với nhiều ứng dụng, việc tối ưu hóa các lệnh gọi LLM đơn lẻ kèm truy xuất (retrieval) và ví dụ trong ngữ cảnh (in-context examples) thường là đủ.

Building block cơ bản — “augmented LLM”: một LLM được tăng cường bằng các khả năng bổ sung như truy xuất (retrieval — tìm thông tin liên quan từ tập nguồn dữ liệu), tool, và bộ nhớ (memory). Những khả năng này cho phép model tự sinh truy vấn tìm kiếm và tự quyết định thông tin nào cần giữ lại.

Các workflow pattern nền tảng được mô tả:

- **Prompt chaining:** chia một tác vụ thành chuỗi các bước, mỗi bước xử lý output của bước trước — ví dụ viết outline tài liệu, kiểm tra outline có đạt tiêu chí không, rồi mới viết tài liệu dựa trên outline.
- **Routing:** phân loại input và định tuyến đến workflow chuyên biệt phía sau — đây chính là vai trò trung tâm của orchestrator trong kiến trúc bạn đang xây (route câu hỏi thuế → sub-agent thuế, câu hỏi đất đai → sub-agent đất đai).
- **Parallelization:** chạy nhiều lệnh gọi LLM song song trên cùng một tác vụ, sau đó tổng hợp kết quả (dạng sectioning — chia nhỏ tác vụ độc lập; hoặc voting — chạy nhiều lần để lấy kết quả đa số/đáng tin hơn).
- **Orchestrator-worker:** một LLM trung tâm động phân rã tác vụ, giao cho các LLM worker, rồi tổng hợp kết quả — đây chính là kiến trúc trung tâm của Giai đoạn 2.
- **Evaluator-optimizer:** một LLM sinh phản hồi, một LLM khác đánh giá và đưa phản hồi (feedback) theo vòng lặp — hữu ích khi có tiêu chí đánh giá rõ ràng và việc tinh chỉnh lặp đi lặp lại mang lại giá trị đo lường được.

Về công cụ (tools): dù xây hệ thống agentic nào, tool gần như luôn là phần quan trọng. Tool cho phép Claude tương tác với các dịch vụ và API bên ngoài bằng cách chỉ rõ cấu trúc và định nghĩa chính xác của chúng trong API.

Ứng dụng thực tế nổi bật được nêu trong bài: agent viết mã (coding agents) đặc biệt hiệu quả vì giải pháp mã nguồn có thể được kiểm chứng qua kiểm thử tự động, agent có thể lặp lại giải pháp dựa trên kết quả kiểm thử làm phản hồi, không gian bài toán được định nghĩa và cấu trúc rõ ràng, và chất lượng output có thể đo lường một cách khách quan. Ứng dụng thứ hai là customer support — kết hợp giao diện chatbot quen thuộc với khả năng tăng cường qua tích hợp tool, phù hợp tự nhiên với agent mở vì tương tác hỗ trợ vốn đi theo luồng hội thoại trong khi cần truy cập thông tin và hành động bên ngoài (lấy dữ liệu khách hàng, lịch sử đơn hàng, tra cứu cơ sở tri thức; thực hiện hành động như hoàn tiền, cập nhật ticket).

Về giám sát con người: một cách tiếp cận hiệu quả là xây các checkpoint nơi agent tạm dừng để con người xem xét — đặc biệt quan trọng trước khi thực hiện các hành động không thể đảo ngược, như phê duyệt giao dịch tài chính hay xóa dữ liệu.

2.2. Reference implementation (code mẫu các pattern)

Nguồn: <https://github.com/anthropics/claude-cookbooks/tree/main/patterns/agents>

Kho mã nguồn tham khảo cho bài “Building Effective Agents”, chứa các cài đặt tối giản (minimal implementations) của các workflow pattern phổ biến đã mô tả ở trên — hữu ích để bắt đầu code thử nghiệm nhanh trước khi tự thiết kế kiến trúc riêng.

3. Giai đoạn 2 — Kiến trúc Multi-agent (Orchestrator-Worker)

3.1. “How we built our multi-agent research system” (Anthropic Engineering)

Nguồn: <https://www.anthropic.com/engineering/multi-agent-research-system>

Đây là tài liệu **quan trọng nhất** cho đúng kiến trúc bạn muốn xây — mô tả chi tiết hệ thống Research của Anthropic, sử dụng kiến trúc multi-agent theo mô hình **orchestrator-worker**: một lead agent điều phối toàn bộ quy trình trong khi giao việc cho các subagent chuyên biệt hoạt động song song.

3.1.1. Luồng hoạt động tổng thể

Khi người dùng gửi một truy vấn, lead agent phân tích truy vấn, xây dựng chiến lược, rồi sinh ra các subagent để khám phá song song các khía cạnh khác nhau. Các subagent hoạt động như những bộ lọc thông minh — lặp đi lặp lại việc dùng tool tìm kiếm để thu thập thông tin, sau đó trả về danh sách kết quả cô đọng cho lead agent để nó tổng hợp thành câu trả lời cuối cùng.

Cách tiếp cận này khác biệt với Retrieval Augmented Generation (RAG) truyền thống — RAG dùng truy xuất tĩnh: lấy một tập các đoạn văn bản (chunk) giống với truy vấn đầu vào nhất rồi dùng các chunk đó để sinh câu trả lời. Cách tiếp cận multi-agent linh hoạt hơn nhiều vì mỗi subagent có thể tự điều chỉnh chiến lược tìm kiếm dựa trên những gì nó khám phá được.

3.1.2. Ba thành phần chính của kiến trúc

- **Lead Researcher agent (điều phối viên chính)**: là bộ điều phối trung tâm. Khi người dùng gửi truy vấn, Lead Researcher phân tích, quyết định chiến lược tổng thể, và ghi lại kế hoạch vào bộ nhớ (memory). Quản lý bộ nhớ ở đây rất quan trọng vì các tác vụ nghiên cứu lớn có thể dễ dàng vượt quá giới hạn token của context window của model — bằng cách lưu kế hoạch, hệ thống tránh mất dấu công việc khi token cạn.
- **Subagent (agent con chuyên biệt)**: được Lead Researcher tạo ra, mỗi subagent được giao một nhiệm vụ cụ thể — chẳng hạn khám phá một công ty cụ thể, kiểm tra một khoảng thời gian nhất định, hoặc tìm hiểu sâu một chi tiết kỹ thuật. Vì các subagent hoạt động song song và duy trì context riêng, chúng có thể tìm kiếm, đánh giá kết quả, và tinh chỉnh truy vấn một cách độc lập mà không can thiệp lẫn nhau.
- **Citation Agent (tác nhân trích dẫn)**: đảm bảo tính chính xác bằng cách gắn kết quả trở lại với nguồn gốc — tạo nên hệ thống vừa mạnh mẽ vừa đáng tin cậy hơn so với cách tiếp cận đơn-agent hoặc truy xuất tĩnh.

Cấu trúc phân tầng này (Lead Researcher → Subagents → Citation Agent) mang lại sự linh hoạt, chiều sâu, và trách nhiệm giải trình rõ ràng: Lead Researcher đảm bảo định hướng và tính nhất quán, subagent cung cấp khả năng khám phá song song và mở rộng quy mô, còn Citation Agent đảm bảo độ chính xác bằng cách gắn kết quả với nguồn.

3.1.3. Nguyên tắc kỹ thuật prompt cho hệ multi-agent

Các agent ban đầu mắc nhiều lỗi: sinh ra tới 50 subagent cho một truy vấn đơn giản, quét web vô tận để tìm các nguồn không tồn tại, và làm phiền lẫn nhau bằng các cập nhật trạng thái thừa thãi. Vì mỗi agent được dẫn dắt bởi một prompt, kỹ thuật viết prompt (prompt engineering) là đòn bẩy chính để cải thiện các hành vi này. Các nguyên tắc rút ra:

- **Nghĩ như agent của bạn (“Think like your agents”)**: để lặp lại/tinh chỉnh prompt, cần hiểu tác động thực sự của nó. Anthropic xây dựng các mô phỏng (simulations) dùng Console với đúng prompt và tool của hệ thống thật, rồi quan sát agent làm việc từng bước một. Điều này lập tức lộ ra các chế độ lỗi: agent tiếp tục tìm kiếm dù đã có đủ kết quả, dùng truy vấn tìm kiếm quá dài dòng, hoặc chọn sai tool.
- **Dạy orchestrator cách giao việc (“Teach the orchestrator how to delegate”)**: cần các mô tả nhiệm vụ chi tiết cho từng subagent — nếu không, agent sẽ trùng lặp công việc

hoặc để lại khoảng trống thông tin quan trọng. Prompt giao việc cần có: mục tiêu rõ ràng, định dạng output, hướng dẫn dùng tool/nguồn nào, và ranh giới nhiệm vụ rõ ràng (ví dụ: “không nghiên cứu X, đó là việc của subagent khác”).

- **Effort-scaling (chia tỉ lệ nỗ lực theo độ khó):** agent gặp khó khăn trong việc đánh giá mức tài nguyên phù hợp cho từng loại truy vấn, dẫn đến hoặc khám phá không đủ hoặc tiêu tốn token quá mức. Đội ngũ đưa ra các quy tắc rõ ràng ngay trong prompt: tra cứu sự thật đơn giản cần 1 agent với 3–10 lần gọi tool; so sánh trực tiếp cần 2–4 subagent với 10–15 lần gọi mỗi agent; nghiên cứu phức tạp có thể dùng hơn 10 subagent với trách nhiệm được phân chia rõ ràng. Những quy tắc này ngăn cả tình trạng đầu tư dưới mức cho truy vấn phức tạp lẫn đầu tư quá mức cho truy vấn đơn giản.
- **Phương pháp nghiên cứu phản chiếu cách con người làm việc:** prompt gồm cả chiến lược bắt đầu bằng truy vấn rộng trước khi thu hẹp trọng tâm, đánh giá chất lượng nguồn, điều chỉnh cách tìm kiếm dựa trên phát hiện, và cân bằng giữa chiều sâu và chiều rộng khám phá.
- **Đánh giá của con người phát hiện những gì tự động hóa bỏ sót:** người kiểm thử phát hiện các trường hợp biên mà eval tự động bỏ sót — ví dụ câu trả lời bị hallucination trên các truy vấn bất thường, lỗi hệ thống, hoặc thiên lệch tinh vi trong việc chọn nguồn. Người kiểm thử thực tế nhận thấy các agent ban đầu liên tục chọn nội dung được tối ưu hóa SEO (content farm) thay vì các nguồn có thẩm quyền nhưng ít được xếp hạng cao hơn như tài liệu học thuật (PDF) hoặc blog cá nhân — việc thêm các heuristic đánh giá chất lượng nguồn vào prompt đã giải quyết vấn đề này.
- **Hệ thống multi-agent có hành vi nổi lên (emergent behavior),** tức phát sinh mà không cần lập trình cụ thể — ví dụ những thay đổi nhỏ ở lead agent có thể thay đổi hành vi của subagent theo cách khó dự đoán. Vì vậy prompt tốt nhất không chỉ là chỉ dẫn nghiêm ngặt, mà là khung làm việc (framework) cho sự hợp tác, định nghĩa cách phân chia lao động và cách tiếp cận giải quyết vấn đề.

3.1.4. Song song hóa và “thinking” có kiểm soát

Anthropic sử dụng **extended thinking** (suy nghĩ mở rộng) và **interleaved thinking** (suy nghĩ xen kẽ) như những “sổ nháp có thể kiểm soát” (controllable scratchpads). Extended thinking cho phép Lead Researcher viết ra lý luận của mình trước khi hành động, chẳng hạn lên kế hoạch dùng tool nào hoặc tạo bao nhiêu subagent. Các subagent cũng lên kế hoạch từng bước, và sau khi nhận kết quả tool, dùng interleaved thinking để đánh giá kết quả, phát hiện khoảng trống, và tinh chỉnh truy vấn tiếp theo. Kiểu lý luận có cấu trúc này cải thiện độ chính xác và hiệu quả.

Các hệ thống ban đầu chạy tìm kiếm tuần tự (lần lượt), rất chậm. Bằng cách thiết kế lại prompt để khuyến khích song song hóa, đội ngũ đạt được tăng tốc đáng kể — Lead Researcher giờ sinh nhiều subagent cùng lúc, và mỗi subagent có thể dùng nhiều tool song song. Điều này giảm thời gian nghiên cứu tới **90%** cho các truy vấn phức tạp, biến việc thu thập thông tin rộng từ hàng giờ xuống còn vài phút.

3.1.5. Bản chất “nén” của tìm kiếm và lý do multi-agent hiệu quả

Bản chất của tìm kiếm là sự **nén (compression)**: chắt lọc insight từ một kho ngữ liệu khổng lồ. Subagent hỗ trợ việc nén này bằng cách hoạt động song song với context window riêng, khám phá các khía cạnh khác nhau của câu hỏi đồng thời trước khi cô đọng các token quan trọng nhất cho lead agent tổng hợp. Mỗi subagent cũng mang lại sự **tách biệt mối quan tâm (separation of concerns)** — tool, prompt, và quỹ đạo khám phá riêng biệt — giúp giảm sự phụ thuộc theo đường đi (path dependency) và cho phép điều tra độc lập, kỷ lưỡng.

Khi trí tuệ đạt đến một ngưỡng nhất định, hệ thống multi-agent trở thành cách quan trọng để mở rộng quy mô hiệu năng. Ví von: dù từng cá nhân con người đã trở nên thông minh hơn trong 100.000 năm qua, xã hội loài người đã trở nên mạnh mẽ hơn theo cấp số nhân trong kỷ nguyên thông tin nhờ trí tuệ tập thể và khả năng phối hợp. Ngay cả các agent có trí tuệ tổng quát cũng gặp giới hạn khi hoạt động đơn lẻ; các nhóm agent có thể hoàn thành nhiều việc hơn nhiều.

3.1.6. Đo lường hiệu năng và chi phí

Hệ thống dùng Claude Opus 4 làm lead agent và Claude Sonnet 4 làm subagent, vượt trội hơn Claude Opus 4 đơn-agent tới **90,2%** trên bộ đánh giá nghiên cứu nội bộ, đặc biệt xuất sắc ở các truy vấn “breadth-first” (cần khám phá nhiều hướng độc lập cùng lúc) — ví dụ xác định thành viên hội đồng quản trị của toàn bộ công ty ngành Công nghệ Thông tin trong S&P 500.

Đổi lại, hệ thống multi-agent tiêu tốn xấp xỉ **15 lần token** so với một cuộc chat thông thường. Kinh tế học vẫn ủng hộ cách tiếp cận này bất chấp mức tiêu thụ token cao hơn nhiều — hiệu năng đạt được biện minh cho chi phí đối với các tác vụ giá trị cao, phức tạp, đòi hỏi kiến thức chuyên biệt hoặc vượt quá giới hạn context của agent đơn lẻ.

3.1.7. Giới hạn của kiến trúc multi-agent

Multi-agent phù hợp nhất với các bài toán có thể chia thành các nhánh nghiên cứu song song, nhưng kém hiệu quả hơn với các tác vụ phụ thuộc chặt chẽ lẫn nhau như lập trình (coding). Domain đòi hỏi tất cả agent chia sẻ cùng một context, hoặc có nhiều phụ thuộc giữa các agent, hiện **không phải** là lựa chọn phù hợp cho hệ thống multi-agent — coding, debug, và phần lớn workflow agentic khác không đáp ứng được điều kiện này.

Nếu subagent B cần kết quả của subagent A để hoàn thành việc của mình, tính song song sẽ suy biến thành thực thi tuần tự tốn kém hơn kèm overhead phối hợp phụ trội — đây là điểm cần cân nhắc kỹ nếu trong hệ thống pháp lý của bạn, câu hỏi về đất đai đôi khi cần tham chiếu chéo tới quy định thuế.

Một ví dụ lỗi thực tế: một subagent điều tra khủng hoảng chip ô tô năm 2021 trong khi hai subagent khác trùng lặp công việc về chuỗi cung ứng hiện tại năm 2025. Cách khắc phục là mô tả nhiệm vụ tốt hơn trong prompt giao việc của orchestrator: mục tiêu rõ ràng, ranh giới rõ ràng, và chỉ dẫn rõ ràng về việc không được nghiên cứu gì.

3.1.8. Mô hình thay thế: swarm/peer-to-peer

Một mô hình kiến trúc khác là **swarm hay peer-to-peer**, nơi các agent giao tiếp trực tiếp với nhau và chia sẻ trạng thái, thường qua một message bus hoặc scratchpad chung. Mô hình swarm linh hoạt nhưng khó lý luận (reason) về hành vi hệ thống. Orchestrator-worker ràng buộc chặt topology: worker không bao giờ nói chuyện trực tiếp với nhau, mọi quyết định về bước tiếp theo đều nằm ở orchestrator.

Quyết định quan trọng nhất trong bất kỳ thiết kế multi-agent nào là **ranh giới cô lập (isolation boundary)**: mỗi subagent cần biết bao nhiêu về việc các subagent khác đang làm? Với hệ thống nghiên cứu, câu trả lời của Anthropic là “gần như không cần biết gì cả”. Mỗi subagent nhận một mô tả nhiệm vụ độc lập, một định dạng output, và một context window hoàn toàn mới — nó không biết các subagent khác tồn tại, không thể phối hợp giữa chừng tác vụ. Đây chính là điều cho phép các subagent chạy song song thực sự và giữ cho context window của lead agent không bị ngập trong các trao đổi chéo.

3.1.9. Vận hành production: checkpoint, retry, đồng bộ hóa

Việc chuyển từ prototype sang sản phẩm đòi hỏi: duy trì trạng thái agent qua các lần tool thất bại, gỡ lỗi các hành vi động bằng khả năng truy vết đầy đủ (traceability), và triển khai cập nhật an toàn bằng “rainbow deployments” (triển khai theo kiểu cầu vồng — chuyển dần lưu lượng để tránh gián đoạn).

Hệ thống hiện hoạt động **đồng bộ (synchronous)** — lead agent chờ mỗi nhóm subagent hoàn thành trước khi tiếp tục — điều này đơn giản hóa việc phối hợp nhưng tạo ra nút thắt cổ chai trong luồng thông tin và ngăn khả năng điều chỉnh/phối hợp theo thời gian thực. Kiến trúc bất đồng bộ (asynchronous) có thể mang lại thêm tính song song nhưng đặt ra thách thức trong việc phối hợp kết quả, tính nhất quán trạng thái, và lan truyền lỗi — được ghi nhận là hướng phát triển tương lai.

Để đánh giá, Anthropic dùng cả **LLM-as-judge** (LLM đóng vai giám khảo) đánh giá tự động theo rubric về độ chính xác, trích dẫn, và chất lượng nguồn, kết hợp với người kiểm thử để bắt các lỗi hallucination, thiên lệch, và nguồn yếu.

3.2. Bài phân tích khoảng cách giữa demo và production

Nguồn: <https://fountaincity.tech/resources/blog/anthropic-multi-agent-blueprint-production/>

Bài viết phân tích sâu về bản thiết kế (blueprint) của Anthropic và những gì môi trường production thực tế cần bổ sung thêm:

- **Hợp đồng bốn phần** cho mỗi subagent (mục tiêu, định dạng output, hướng dẫn tool/nguồn, ranh giới nhiệm vụ) — bỏ sót bất kỳ phần nào, subagent sẽ đi lệch hướng; không phải vì model hoạt động kém, mà vì orchestrator đã không chỉ rõ đủ để nó biết “hoàn thành” nghĩa là gì.
- **Effort-scaling là một phần của hợp đồng**: nếu không có quy tắc rõ ràng, lead agent sẽ tự mở rộng quy mô quá mức — sinh subagent cho những bài toán mà một lệnh gọi đơn lẻ đã đủ trả lời — khiến hệ số chi phí nhân lên không kiểm soát.
- **Tool ergonomics (tính tiện dụng của tool)**: hợp đồng chỉ tốt bằng bề mặt tool mà nó tro tới. Thứ tự triển khai đề xuất: làm đúng hợp đồng bốn phần trước → nhúng quy tắc effort-scaling vào prompt orchestrator → kiểm toán mô tả tool → mới lặp lại các phần khác.
- **Giới hạn context**: bản năng thông thường khi gặp giới hạn context là yêu cầu một context window lớn hơn. Kiến trúc của Anthropic làm ngược lại — lead researcher lưu kế hoạch ra bộ nhớ trước khi context đầy, vì quá 200.000 token context có thể bị cắt và kế hoạch cần sống sót qua đó. Lựa chọn kiến trúc là **externalize sớm** (đưa trạng thái ra ngoài sớm), không phải chạy theo context window lớn hơn.
- **Mẫu artifact (artifact pattern)**: thay vì subagent báo cáo phát hiện qua kiểu trả lời như chat (dài, mất mát thông tin, tốn token của lead agent), chúng ghi vào một filesystem chia sẻ và trả về một tham chiếu nhẹ. Lead agent không cần đọc lại từng chi tiết — nó nhận một con trỏ (pointer) và chỉ lấy những gì cần.
- **Khoảng trống mà blueprint để lại cho người đọc tự giải quyết**: (1) trạng thái chạy dài xuyên suốt nhiều phiên — hệ thống Research của Anthropic bị giới hạn trong một phiên (session-bounded), trong khi agent production thường cần hoạt động qua nhiều ngày/tuần (một pipeline nội dung theo dõi bài viết mới, một agent vận hành giám sát hệ thống liên tục, một agent tích hợp xử lý sự kiện khi chúng đến); (2) xử lý khi một subagent thất bại giữa chừng quá trình điều phối.
- Bối cảnh nghiên cứu của Anthropic không có trần chi phí hằng ngày cố định, không có SLA độ chính xác cứng, không có ngân sách phản hồi dưới một giây, không có ngưỡng tỷ lệ lỗi gắn với doanh thu. Phần lớn hệ thống production có ít nhất một, thường là cả bốn ràng buộc này — và các quyết định kiến trúc dưới áp lực đó thường không giống mặc định của blueprint.

3.3. “Building Effective AI Agents: Architecture Patterns and Implementation Frameworks” (whitepaper)

Nguồn: <https://resources.anthropic.com/building-effective-ai-agents>

Whitepaper chia sẻ các thực hành và hướng dẫn tốt nhất để xây dựng hệ thống agentic sẵn sàng cho production, bao gồm thiết kế đơn-agent, điều phối multi-agent, và workflow agentic, kèm ví dụ thực tế từ các công ty như Coinbase, Intercom, và Thomson Reuters đang xây dựng và triển khai các hệ thống này ở quy mô lớn.

Nội dung chính bao gồm: cách lựa chọn giữa kiến trúc đơn-agent, multi-agent, và dựa-trên-workflow; khi nào dùng pattern sequential (tuần tự), parallel (song song), và evaluator-optimizer; cách khớp độ phức tạp kỹ thuật với giá trị kinh doanh bằng khung quyết định (decision framework); và các thực hành tốt nhất cho quản lý context, thiết kế mô-đun, và sử dụng Skills.

Nguyên tắc chủ đạo về sự tiến hóa kiến trúc: kiến trúc nên tiến hóa theo nhu cầu. Bắt đầu đơn giản, đo lường mọi thứ, chỉ thêm độ phức tạp khi nó mang lại giá trị đo lường được. Kiến trúc tốt nhất là kiến trúc đơn giản nhất đáp ứng được yêu cầu hiện tại — nhưng cũng không bị bó buộc vào các pattern đơn giản mãi mãi: khi nhu cầu kinh doanh biện minh cho độ phức tạp tăng thêm, việc kết hợp các pattern một cách chiến lược có thể mở khóa những khả năng mà cách tiếp cận đơn lẻ không thể đạt được.

Khuyến nghị bắt đầu: bắt đầu với các agent đơn mục đích, làm tốt một việc, sau đó dần phát triển chúng thành các hệ thống tinh vi hơn khi yêu cầu phát triển. Hệ thống đơn giản rõ hơn để

vận hành (ít token hơn, ít tính toán hơn), dễ gỡ lỗi hơn khi có sự cố, và cho các chỉ số rõ ràng thực sự gắn với giá trị kinh doanh.

Về việc chọn model: nếu xây dựng khung coding multi-agent hoặc phân tích tài chính phức tạp, cần model mạnh nhất có sẵn. Nhưng nếu xử lý hàng nghìn ticket hỗ trợ khách hàng đơn giản hoặc trích xuất dữ liệu từ biểu mẫu, một model nhẹ, nhanh hơn sẽ hoàn thành công việc tốt như vậy — điều này khớp trực tiếp với ý tưởng orchestrator “flash-lite” của bạn.

Thiết kế mô-đun (modular design): lĩnh vực này thay đổi nhanh; các khả năng và tính năng mới xuất hiện thường xuyên. Thiết kế hệ thống theo hướng mô-đun để có thể tiến hóa khả năng của agent mà không cần thiết kế lại toàn bộ hạ tầng — kiến trúc nên uốn theo sự tiến bộ, không gãy vỡ dưới áp lực đó. Ví dụ mô-đun hóa agent có thể tận dụng pattern composition, nơi prompt được định nghĩa trong các file/thư viện cấu hình tập trung, và mở rộng khả năng bằng Agent Skills — các gói năng lực mô-đun mà agent có thể tải khi cần thay vì nhúng trực tiếp vào prompt.

Xây hệ thống có khả năng tự giải thích (observable): ứng dụng AI thường hoạt động như hộp đen, và agent bổ sung thêm nhiều tầng phức tạp. Ngoài các thực hành tiêu chuẩn như logging có cấu trúc, giám sát tập trung, và distributed tracing, ứng dụng AI đặt ra các thách thức quan sát (observability) độc đáo mà công cụ giám sát hiệu năng ứng dụng truyền thống không được thiết kế để xử lý. Trong kiến trúc multi-agent, nơi quyết định của agent nhân lên nhiều lần, việc triển khai tracing bắt buộc không chỉ hành vi từng agent riêng lẻ mà cả mẫu quyết định và cấu trúc tương tác giữa các agent là điều thiết yếu để chẩn đoán khi phối hợp thất bại.

Về kinh tế token: dù hệ thống multi-agent tiêu thụ token nhiều hơn đáng kể so với tương tác đơn, lợi ích hiệu năng biện minh cho chi phí đối với các tác vụ giá trị cao, phức tạp cần kiến thức chuyên biệt hoặc vượt giới hạn context của agent đơn.

4. Giai đoạn 3 — Context Engineering

4.1. “Effective context engineering for AI agents” (Anthropic Engineering, 09/2025)

Nguồn: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

Bài viết khám phá “nghệ thuật đang nổi lên” của context engineering, cung cấp một mô hình tinh chỉnh cho việc xây dựng agent dễ điều khiển (steerable) và hiệu quả. Anthropic xem context engineering là bước tiến tự nhiên của prompt engineering.

Định nghĩa: Context (ngữ cảnh) là tập hợp các token được đưa vào khi lấy mẫu (sample) từ một large language model. Bài toán kỹ thuật ở đây là tối ưu hóa tính hữu ích của các token đó trong giới hạn vốn có của LLM, nhằm đạt được kết quả mong muốn một cách nhất quán.

Prompt engineering vs context engineering: prompt engineering đề cập đến các phương pháp viết và tổ chức chỉ dẫn LLM để đạt kết quả tối ưu (đặc biệt là system prompt), phù hợp nhất khi phần lớn use case là các tác vụ phân loại một lần (one-shot) hoặc sinh văn bản. Context engineering đề cập đến tập hợp các chiến lược để chọn lọc và duy trì tập token (thông tin) tối ưu trong suốt quá trình suy luận LLM, bao gồm tất cả các thông tin khác có thể xuất hiện ngoài phạm vi prompt. Khi tiến tới xây dựng agent có năng lực hơn, hoạt động qua nhiều lượt suy luận và khung thời gian dài hơn, cần các chiến lược để quản lý **toàn bộ trạng thái context**: system instructions, tool, Model Context Protocol (MCP), dữ liệu ngoài, lịch sử tin nhắn, v.v.

Một agent chạy trong vòng lặp sinh ra ngày càng nhiều dữ liệu có thể liên quan cho lượt suy luận tiếp theo, và thông tin này phải được tinh chỉnh theo chu kỳ. Context engineering là nghệ thuật và khoa học của việc chọn lọc những gì sẽ đi vào context window giới hạn từ vũ trụ thông tin khả dĩ đang không ngừng tiến hóa đó. Khác với nhiệm vụ rời rạc là viết một prompt, context engineering mang tính lặp đi lặp lại, và giai đoạn chọn lọc diễn ra mỗi lần quyết định thứ gì sẽ được đưa vào model.

Nguyên tắc “độ cao” (altitude) cho system prompt: system prompt cần cực kỳ rõ ràng, dùng ngôn ngữ đơn giản, trực tiếp, trình bày ý tưởng ở “độ cao” phù hợp cho agent. Độ cao phù hợp nằm giữa hai chế độ lỗi phổ biến: - **Cực đoạn thứ nhất:** các kỹ sư hard-code logic phức tạp, dễ vỡ vào prompt để ép agent hành xử chính xác — cách này tạo ra sự mong manh và tăng độ phức tạp bảo trì theo thời gian. - **Cực đoạn thứ hai:** các kỹ sư đưa hướng dẫn mơ hồ, cấp cao, không cho model tín hiệu cụ thể về output mong muốn, hoặc giả định sai rằng có ngữ cảnh chung đã được chia sẻ.

Độ cao tối ưu cân bằng giữa hai cực này: đủ cụ thể để định hướng hành vi hiệu quả, nhưng đủ linh hoạt để cung cấp cho model các heuristic mạnh để tự định hướng hành vi.

Context rot (sự suy thoái ngữ cảnh): nghiên cứu benchmark kiểu “needle-in-a-haystack” (tìm kim trong đống rơm) phát hiện ra khái niệm context rot — khi số token trong context window tăng lên, khả năng nhớ lại chính xác thông tin từ context đó của model sẽ giảm xuống. Đây là một thực tế kiến trúc, không phải lỗi — kiến trúc transformer tạo ra các quan hệ n^2 theo cặp giữa các token: ở 10.000 token là 100 triệu quan hệ cần theo dõi, ở 100.000 token là 10 tỷ quan hệ. Giống bộ nhớ làm việc của con người, LLM mất tập trung khi bị quá tải — khác biệt là con người tự nhiên lọc và ưu tiên thông tin, còn với AI agent, cần kỹ sư thiết kế việc lọc đó thay cho chúng.

Nguyên tắc thực hành cốt lõi: tìm **tập token tín hiệu cao nhỏ nhất** có khả năng tối đa hóa xác suất đạt được kết quả mong muốn.

Ba nguyên thủy (primitives) chính để quản lý context dài hạn (mô tả chi tiết trong cookbook đi kèm — xem 4.2): - **Compaction:** chưng cất nội dung của context window thành một bản tóm tắt độ trung thực cao, cho phép agent tiếp tục với suy giảm hiệu năng tối thiểu khi hội thoại dài ra. - **Tool-result clearing:** xử lý tình trạng phình to do bản thân việc dùng tool gây ra. - **Memory (bộ nhớ):** agent thường xuyên viết ghi chú ra bộ nhớ ngoài (ví dụ file NOTES.md hoặc memory tool) và kéo chúng trở lại context khi cần — cung cấp trạng thái bền vững với overhead tối thiểu. Kỷ luật cần thiết: bộ nhớ bền vững chỉ nên chứa thông tin tiếp tục ràng buộc việc suy luận trong tương lai — sở thích lâu dài, quyết định, cách tiếp cận đã thất bại. Lưu quá nhiều sẽ tạo ra ô nhiễm context bền vững. “Tính mất mát (lossiness) là một dải phổ, không phải nhị phân.” - Bên cạnh ba nguyên thủy trên, còn có chiến lược cốt lõi khác: **tận**

dụng subagent — khi quản lý bộ nhớ đơn-agent chạm giới hạn, việc phân phối công việc ra các subagent chuyên biệt là bước tiếp theo. Thay vì một agent duy trì trạng thái xuyên suốt toàn bộ dự án, subagent xử lý các tác vụ tập trung với context window sạch, cô lập. Agent chính điều phối qua một kế hoạch cấp cao; subagent thực hiện công việc sâu và trả về các bản tóm tắt cô đọng, nhằm mục tiêu 1.000–2.000 token thay vì toàn bộ dấu vết (trace) đầy đủ.

Về retrieval control: retrieval do agent tự kiểm soát — nơi model tự gọi retrieval như một tool khi nó nhận ra nhu cầu — tạo ra các truy vấn đúng trọng tâm vào đúng thời điểm. Áp dụng RAG cho mô tả tool cải thiện độ chính xác chọn tool lên gấp 3 lần (theo báo cáo của LangChain). Mỗi schema tool chiếm dụng context; một bộ tool phình to dẫn đến “context confusion” — model hallucinate tham số hoặc chọn sai tool hoàn toàn. Lựa chọn tool về bản chất là một bài toán ngân sách context. Khuyến nghị: giữ 3–5 tool dùng nhiều nhất luôn được nạp sẵn, và dùng khám phá động (dynamic discovery) cho các bộ tool từ 10 trở lên. Triển khai không gian hành động phân tầng: một tập nhỏ tool nguyên tử cốt lõi, với các thao tác phức tạp được xử lý qua tool thực thi tổng quát (như bash/shell) thay vì nhân rộng thêm nhiều tool chuyên biệt. Một phép thử thực tế: nếu một kỹ sư con người không thể xác định chắc chắn nên dùng tool nào trong một tình huống nhất định, agent cũng không thể được kỳ vọng làm điều đó.

4.2. Cookbook thực hành: compaction, tool-result clearing, memory

Nguồn: <https://platform.claude.com/cookbook/tool-use-context-engineering-context-engineering-tools>

Cookbook đóng khung vấn đề như một bài toán tài nguyên: context là hữu hạn với lợi nhuận biên giảm dần, và kỷ luật cốt lõi là tìm tập token tín hiệu cao nhỏ nhất tối đa hóa khả năng đạt kết quả mong muốn.

Thách thức phổ biến khi xây agent chạy dài (long-horizon agents) là quản lý context: kết quả tool, lý luận của chính model, và tin nhắn người dùng đều tích lũy dần, cuối cùng hoặc chạm giới hạn token hoặc bắt đầu phải trả tiền cho context không còn giúp ích gì. Cookbook tập trung vào ba kỹ thuật: compaction, tool-result clearing, và memory — cả ba đều có hỗ trợ first-party trong API, nên có thể áp dụng mà không cần xây hạ tầng điều phối riêng.

Cookbook minh họa qua một bộ tài liệu tổng hợp (khoảng 320.000 token tổng cộng, chia thành 8 tài liệu tổng hợp ~40.000 token mỗi tài liệu) với tác vụ đọc theo hai đợt (batch) — cấu trúc này tạo ra một quỹ đạo context vượt qua ngưỡng kích hoạt compaction ở đợt đầu và vượt mốc tham chiếu 200.000 token ở đợt hai, để hiệu ứng của mỗi nguyên thủy hiện rõ trong cùng một lần chạy. Nếu không có quản lý context, context của agent sẽ phình tới hàng trăm nghìn token giữa tác vụ, và vì công việc trải dài qua nhiều phiên, ngay cả một lần chạy đã hoàn thành cũng bắt đầu phiên tiếp theo mà không có ký ức gì về những gì đã học được (nếu không có cơ chế lưu).

4.3. “Equipping agents for the real world with Agent Skills” (Anthropic Engineering)

Nguồn: <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>

Đây là bài viết trực tiếp mô tả cơ chế **Skill** mà bạn muốn gắn cho từng loại vấn đề pháp lý. Claude vốn mạnh, nhưng công việc thực tế đòi hỏi tri thức quy trình (procedural knowledge) và ngữ cảnh tổ chức. Agent Skills là cách mới để xây agent chuyên biệt bằng file và thư mục.

Khi khả năng của model cải thiện, ta có thể xây các agent đa dụng tương tác với môi trường điện toán đầy đủ tính năng. Claude Code chẳng hạn có thể hoàn thành các tác vụ phức tạp trên nhiều lĩnh vực bằng cách thực thi mã cục bộ và hệ thống file. Nhưng khi các agent này trở nên mạnh mẽ hơn, cần những cách mô-đun hơn, mở rộng quy mô tốt hơn, và dễ chuyển đổi (portable) hơn để trang bị cho chúng chuyên môn theo từng lĩnh vực cụ thể. Điều này dẫn tới việc tạo ra Agent Skills: các thư mục có tổ chức chứa chỉ dẫn, script, và tài nguyên mà agent có thể khám phá và tải một cách động để làm tốt hơn ở các tác vụ cụ thể. Skill mở rộng khả năng của Claude bằng cách đóng gói chuyên môn thành tài nguyên có thể ghép nối (composable), biến agent đa dụng thành agent chuyên biệt.

Cấu trúc kỹ thuật: một skill là một thư mục chứa file `SKILL.md`, phải bắt đầu bằng YAML frontmatter chứa metadata bắt buộc: `name` và `description`. Khi khởi động, agent tải trước tên

và mô tả của mọi skill đã cài đặt vào system prompt. Metadata này là tầng đầu tiên của **progressive disclosure (tiết lộ dần dần)**: nó cung cấp đủ thông tin để Claude biết khi nào nên dùng từng skill mà không cần nạp toàn bộ nội dung của nó vào context.

Progressive disclosure là nguyên tắc thiết kế cốt lõi làm cho Agent Skills linh hoạt và mở rộng quy mô được. Giống như một cuốn sổ tay được tổ chức tốt bắt đầu bằng mục lục, rồi đến các chương cụ thể, và cuối cùng là phụ lục chi tiết, skill cho phép Claude tải thông tin chỉ khi cần: agent có hệ thống file và tool thực thi mà không cần đọc toàn bộ skill vào context window khi làm việc trên một tác vụ cụ thể. Điều này có nghĩa dung lượng có thể đóng gói vào một skill về mặt hiệu quả là **vô hạn**. Skill được kích hoạt trong context window thông qua system prompt: ban đầu, context window chỉ chứa system prompt cốt lõi và metadata của mỗi skill đã cài, cùng tin nhắn ban đầu của người dùng; Claude kích hoạt việc đọc chi tiết hơn khi cần.

Skill có thể chứa cả code để Claude thực thi như tool, tùy theo bản chất của tác vụ.

Hướng dẫn xây dựng và kiểm thử skill: - **Bắt đầu bằng đánh giá:** xác định các khoảng trống cụ thể trong năng lực của agent bằng cách chạy chúng trên các tác vụ đại diện, quan sát nơi chúng gặp khó khăn hoặc cần thêm ngữ cảnh. Sau đó xây skill dần để lấp các khoảng trống này. - **Cấu trúc để mở rộng quy mô:** khi file SKILL.md trở nên cồng kềnh, chia nội dung thành các file riêng và tham chiếu tới chúng. Nếu một số ngữ cảnh loại trừ lẫn nhau hoặc hiếm khi dùng cùng nhau, giữ các đường dẫn riêng biệt sẽ giảm mức tiêu thụ token. - Code có thể vừa là tool thực thi, vừa là tài liệu tham khảo. Cần làm rõ liệu Claude nên chạy trực tiếp script hay đọc nó vào context như tài liệu tham khảo.

4.4. LangChain — “Context Engineering” (góc nhìn bổ sung)

Nguồn: <https://www.langchain.com/blog/context-engineering-for-agents>

Bài viết nhóm các chiến lược phổ biến cho context engineering của agent thành bốn nhóm — **write, select, compress, và isolate** — và đưa ra ví dụ cho từng nhóm từ việc xem xét các sản phẩm và bài báo agent phổ biến.

- **Write (ghi):** nghĩa là lưu ngữ cảnh ra ngoài context window để hỗ trợ agent hoàn thành tác vụ. Khi con người giải quyết tác vụ, ta ghi chú và ghi nhớ mọi thứ cho các tác vụ liên quan trong tương lai — agent cũng đang có được khả năng này. Ghi chú qua một “scratchpad” là một cách để duy trì thông tin trong khi agent đang thực hiện tác vụ — ý tưởng là lưu thông tin ra ngoài context window để nó sẵn sàng khi cần. Hệ thống nghiên cứu multi-agent của Anthropic minh họa rõ ràng cho ví dụ này: LeadResearcher bắt đầu bằng cách suy nghĩ qua cách tiếp cận và lưu kế hoạch của nó vào Memory để duy trì ngữ cảnh, vì nếu context window vượt quá 200.000 token nó sẽ bị cắt bớt, và việc giữ lại kế hoạch là quan trọng.
- **Select (chọn lọc):** chọn thông tin liên quan nhất để đưa vào context tại mỗi bước.
- **Compress (nén):** nén thông tin để giữ lại phần tín hiệu cao, loại bỏ phần dư thừa.
- **Isolate (cô lập):** hệ thống nghiên cứu multi-agent của Anthropic cũng minh họa cho chiến lược này — nhiều agent với context bị cô lập vượt trội hơn agent đơn, phần lớn vì mỗi context window của subagent có thể được phân bổ cho một sub-task hẹp hơn. Như bài blog gốc nói: subagent hoạt động song song với context window riêng, khám phá các khía cạnh khác nhau của câu hỏi đồng thời. Đương nhiên, thách thức đi kèm với multi-agent bao gồm mức tiêu thụ token (tới 15 lần so với chat, theo báo cáo của Anthropic), nhu cầu về kỹ thuật prompt cẩn thận để lên kế hoạch công việc cho subagent, và việc phối hợp giữa các subagent.

Framework LLM được ví như một loại hệ điều hành mới (theo Andrej Karpathy): LLM giống CPU và context window của nó giống RAM, đóng vai trò bộ nhớ làm việc của model. Giống RAM, context window của LLM có dung lượng hạn chế để xử lý các nguồn ngữ cảnh khác nhau. Và giống cách một hệ điều hành chọn lọc những gì phù hợp vào RAM của CPU, ta có thể nghĩ về “context engineering” đóng vai trò tương tự.

4.5. “Writing effective tools for AI agents — using AI agents” (Anthropic Engineering)

Nguồn: <https://www.anthropic.com/engineering/writing-tools-for-agents>

Để xây tool hiệu quả cho agent, cần định hướng lại thực hành phát triển phần mềm từ các pattern dự đoán được, xác định (deterministic) sang các pattern không xác định (non-deterministic). Qua quá trình lặp lại, dựa trên đánh giá đã mô tả trong bài, đã xác định được các pattern nhất quán về điều làm cho tool thành công: tool hiệu quả được định nghĩa rõ ràng và có chủ đích, sử dụng ngữ cảnh agent một cách hợp lý, có thể kết hợp cùng nhau trong nhiều workflow khác nhau, và cho phép agent giải quyết trực quan các tác vụ thực tế.

Namespacing (đặt tên theo không gian tên): agent AI của bạn có khả năng tiếp cận hàng chục MCP server và hàng trăm tool khác nhau — bao gồm cả tool do các nhà phát triển khác xây dựng. Khi tool chồng chéo về chức năng hoặc có mục đích mơ hồ, agent có thể bị nhầm lẫn không biết dùng cái nào. Namespacing (nhóm các tool liên quan dưới tiền tố chung) có thể giúp phân định ranh giới giữa nhiều tool; MCP client đôi khi làm điều này mặc định. Ví dụ, đặt tên theo dịch vụ (`asana_search`, `jira_search`) và theo tài nguyên (`asana_projects_search`, `asana_users_search`) có thể giúp agent chọn đúng tool vào đúng thời điểm. Anthropic nhận thấy việc chọn giữa namespacing theo tiền tố và theo hậu tố có ảnh hưởng không nhỏ đến các đánh giá về việc dùng tool.

Nguyên tắc gộp tool: quá nhiều tool hoặc tool chồng chéo có thể khiến agent xao nhãng khỏi việc theo đuổi chiến lược hiệu quả. Lập kế hoạch cẩn thận, có chọn lọc về tool cần xây (hoặc không xây) thực sự đáng giá. Nên xây một vài tool có chủ đích nhằm vào các workflow tác động lớn, khớp với các tác vụ đánh giá của bạn, rồi mở rộng dần từ đó. Cách tiếp cận tốt hơn (cho cả agent lẫn con người) là bỏ qua thẳng đến trang liên quan trước (chẳng hạn tìm theo thứ tự chữ cái) thay vì liệt kê toàn bộ. Ví dụ với trường hợp số địa chỉ, có thể chọn triển khai tool `search_contacts` hoặc `message_contact` thay vì tool `list_contacts`. Tool có thể hợp nhất chức năng, xử lý tiềm năng nhiều thao tác rời rạc (hoặc lệnh gọi API) dưới một lớp vỏ duy nhất — ví dụ, thay vì triển khai `list_users`, `list_events`, và `create_event`, hãy cân nhắc triển khai một tool `schedule_event` tìm khoảng trống lịch và đặt lịch cùng lúc. Thay vì triển khai tool `read_logs`, cân nhắc triển khai `search_logs` chỉ trả về các dòng log liên quan cùng một phần ngữ cảnh xung quanh.

Quy trình đánh giá tool: có thể truyền tool trực tiếp vào lệnh gọi Anthropic API để kiểm thử theo chương trình. Tự kiểm thử tool để xác định các điểm gồ ghề. Thu thập phản hồi từ người dùng để xây trực giác quanh các use-case và prompt mà bạn kỳ vọng tool của mình hỗ trợ. Tiếp theo, cần đo lường mức độ Claude sử dụng tốt tool của bạn bằng cách chạy một đánh giá (evaluation). Bắt đầu bằng việc sinh nhiều tác vụ đánh giá, bám sát các use-case thực tế. Khuyến nghị hợp tác với một agent để giúp phân tích kết quả và xác định cách cải thiện tool.

5. Giai đoạn 4 — Loop Engineering

5.1. Bối cảnh xuất hiện thuật ngữ

“Loop engineering” là thuật ngữ rất mới, bùng nổ trong tuần thứ hai của tháng 6/2026, đánh dấu một sự tái tổ chức trong cách nhiều người nói về việc làm việc với AI: ngừng gõ prompt cho coding agent theo từng bước, và bắt đầu thiết kế **vòng lặp (loop)** tự động prompt cho nó.

Thuật ngữ được phổ biến bởi **Addy Osmani** (kỹ sư trưởng tại Google Chrome), tổng hợp ý tưởng từ **Boris Cherny** (Anthropic — người xây dựng Claude Code) và **Peter Steinberger** (nhà sáng lập PSPDFKit/Nutrient). Cách đóng khung này lan truyền qua blog của Osmani, một bài đăng trên Substack, và một kho GitHub tham khảo nhanh chóng trở thành chuẩn cộng đồng.

Peter Steinberger lập luận rằng kỹ năng thực sự đã dịch chuyển từ việc prompt agent sang thiết kế vòng lặp cho chúng, và Osmani đặt tên và cấu trúc hóa thực hành này trong một bài luận có tựa “Loop Engineering” ngay ngày hôm sau. Boris Cherny, từ phía nhà cung cấp (vendor), nói điều tương tự: công việc của anh giờ là viết các vòng lặp điều khiển Claude, chứ không phải prompt trực tiếp cho model.

5.2. Định nghĩa và bản chất

Loop engineering là thực hành thiết kế các hệ thống prompt, xác minh (verify), thử lại (retry), và dừng (stop) agent AI — thay vì tự tay prompt agent theo từng lượt.

Prompt engineering tối ưu hóa **một tương tác đơn lẻ** với model. Loop engineering tối ưu hóa **hệ thống xung quanh nhiều tương tác**: điều gì kích hoạt agent, ngữ cảnh nó nhận được, tool nó được phép dùng, cách output được xác minh, trạng thái được lưu ở đâu, và khi nào hệ thống dừng lại.

Một vòng lặp (loop) là một hệ thống trao cho AI agent một mục tiêu, để nó hành động, kiểm tra kết quả, cập nhật trạng thái, và quyết định có nên tiếp tục, thử lại, dừng, hay leo thang (escalate) hay không. Điều đó nghe có vẻ đơn giản. Phần khó không nằm ở bản thân vòng lặp — mà nằm ở **bộ xác minh (verifier)**.

Nói theo cách khác: nơi prompt engineering hỏi “Tôi nên nói gì để nhận được output tốt nhất?”, loop engineering hỏi “Tôi nên xây hệ thống gì để agent tự tìm việc, thực hiện, xác minh, và ghi nhớ những gì đã làm — mà không cần tôi ở trong vòng lặp chút nào?”

5.3. Andrew Ng — góc nhìn về ba vòng lặp lồng nhau (30/6/2026)

Andrew Ng bổ sung một góc nhìn về xây dựng sản phẩm: các vòng lặp coding agentic bên trong (tính theo phút) được lồng bên trong vòng phản hồi của nhà phát triển (tính theo giờ) và phản hồi người dùng bên ngoài (tính theo ngày), khi đưa phần mềm từ 0 lên 1.

5.4. Lịch sử “Ralph loop” — tiền thân của loop engineering

Trước khi ai gọi nó là “loop engineering”, đã có kỹ thuật **Ralph**. Đầu năm 2026, Geoffrey Huntley mô tả việc chạy một coding agent bên trong một vòng lặp `while` đơn giản: cấp cho agent cùng một prompt đối chiếu với một đặc tả (spec) đã viết sẵn, để nó chọn một tác vụ và triển khai, rồi khởi tạo một instance mới và cấp lại chính prompt đó. Lặp lại đến khi công việc hoàn tất. Ông đặt tên kỹ thuật này theo nhân vật Ralph Wiggum trong phim hoạt hình The Simpsons, vì kỹ thuật này — theo lời ông — “đơn giản một cách tất định trong một thế giới khó đoán”. Nghe có vẻ ngớ ngẩn nhưng lại hiệu quả.

5.5. Framework thực hành: “Operator Loop Stack”

Nguồn: <https://www.jacknjoroge.com/loop-engineering/>

Framework thực hành gồm 5 thành phần để quyết định liệu một vòng lặp agent AI có an toàn để chạy hay không:

1. **Harness (khung vận hành)**: lớp bao bọc điều khiển việc gọi agent.

2. **Loop contract (hợp đồng vòng lặp):** định nghĩa mục tiêu, điều kiện dừng, và ràng buộc của vòng lặp.
3. **State layer (tầng trạng thái):** nơi lưu trạng thái xuyên suốt các lần lặp.
4. **Checker (bộ kiểm tra/verifier):** xác minh kết quả trước khi cho phép tiếp tục.
5. **Human checkpoint (điểm kiểm tra của con người):** điểm dừng để con người xem xét trước các hành động quan trọng.

Tài liệu này cũng cung cấp một “Loop Engineering Starter Kit” gồm: checklist đánh giá mức độ sẵn sàng cho vòng lặp, mẫu hợp đồng vòng lặp (loop contract template), bảng quyết định “/goal vs /loop”, và mẫu prompt verifier kiểu maker/checker.

5.6. Bốn tầng tiến hóa: Prompt → Context → Harness → Loop Engineering

Nguồn: <https://tosea.ai/blog/loop-engineering-ai-agents-complete-guide-2026>

Guide này truy vết nguồn gốc của loop engineering như tầng mới nhất trong một tiến trình bốn bước: từ prompt engineering, đến context engineering, đến harness engineering, rồi đến loop engineering. Bài học mà loop engineering dạy cho các coding agent cũng áp dụng gọn gàng cho các workflow khác (ví dụ công việc thuyết trình/slide): giá trị không nằm ở một output đơn lẻ thông minh, mà nằm ở một chu trình giữ cho output đúng.

Loop engineering, theo định nghĩa này, là việc thiết kế hệ thống chạy một AI agent theo một chu kỳ lặp lại — **hành động, quan sát, quyết định, lặp lại** — thay vì tự tay prompt agent từng bước một. Bạn định nghĩa một mục tiêu và một điều kiện dừng; vòng lặp tự lo phần lặp lại.

5.7. Verifier là nút thắt cổ chai, không phải model

Nguồn: <https://www.aibuilderclub.com/blog/loop-engineering-guide-2026>

Loop engineering là kỷ luật thiết kế vòng lặp mà agent chạy bên trong — nó làm gì giữa các lần gọi tool, khi nào tự kiểm tra công việc, và cách nó quyết định đã “xong” — thay vì viết tay từng prompt. Đây được xem là “người kế nhiệm” của prompt engineering trong năm 2026: model giờ tự viết prompt cho chính nó. Kỹ năng khan hiếm là định nghĩa “tốt” và “xong” nghĩa là gì — và điều mà hầu hết bài giải thích bỏ sót là: trong bất kỳ vòng lặp nào, **verifier mới là nút thắt cổ chai, không phải model**.

Nguồn này cũng tổng hợp một số góc nhìn phản biện đáng chú ý về xu hướng này: - Góc nhìn hoài nghi (PostHog, thread Hacker News): liệu “vòng lặp while” có thực sự mới, và phê bình về mức tiêu thụ token (Ed Zitron) cùng ví dụ về trần chi phí thực tế (một công ty giới hạn 1.500 USD/tháng cho việc này). - “Loop engineering, thuật ngữ thời thượng mới nhất của AI, vẫn cần con người trong vòng lặp” (The Register): kiểm tra thực tế về mức độ tự trị — các lần chạy thực tế vẫn cần con người điều khiển nhiều hơn những gì demo thể hiện. - Phê bình về thiên lệch lấy mẫu: dữ liệu telemetry của nhà cung cấp (vendor) đến từ những người đã dùng sản phẩm đó — cần cẩn trọng khi khái quát hóa hiệu năng.

5.8. Ứng dụng: khi nào cần loop engineering, khi nào không

Nguồn: <https://www.mindstudio.ai/blog/what-is-loop-engineering-ai-coding-agents>

Đây là kiến trúc nền tảng phía sau mọi coding agent nghiêm túc hiện nay, từ Claude Code đến các workflow agentic tự xây dựng. Một số nguyên tắc thực hành được rút ra từ hệ sinh thái này:

- Dùng 5 sub-agent chuyên gia cho việc nghiên cứu tạo ra kết quả tốt hơn so với 100+ agent chạy song song không kiểm soát — nhấn mạnh lại nguyên tắc effort-scaling đã nêu ở Giai đoạn 2.
- Workflow AI tốt nhất không hoàn toàn tự động — cần xác định 2–3 điểm kiểm tra nơi con người xem xét để tránh sai sót tốn kém và “AI slop” (kết quả AI chất lượng thấp tràn lan).
- Trước khi nâng cấp model của một AI agent, cần kiểm toán: nó đọc gì, nó có thể chạm vào gì, nhiệm vụ của nó là gì, nó cung cấp bằng chứng gì, và liệu nó có còn mang lại giá trị hay không.

6. Tài liệu bổ trợ: quản trị agent, containment, human-agent teams

6.1. “Scaling Managed Agents: Decoupling the brain from the hands” (Anthropic Engineering)

Nguồn: <https://www.anthropic.com/engineering/managed-agents>

Một chủ đề xuyên suốt trên Engineering Blog của Anthropic là cách xây agent hiệu quả và thiết kế “harness” (khung vận hành) cho công việc chạy dài hạn. Một sợi chỉ chung xuyên suốt các bài viết này: harness mã hóa các giả định về những gì Claude *không* tự làm được. Nhưng những giả định đó cần được thường xuyên đặt lại câu hỏi, vì chúng có thể lỗi thời khi model cải thiện.

Ví dụ điển hình — “context anxiety”: trong công việc trước đây, Anthropic phát hiện Claude Sonnet 4.5 có xu hướng kết thúc tác vụ sớm khi nó “cảm nhận” giới hạn context đang đến gần — một hành vi đôi khi được gọi là “context anxiety” (lo lắng về ngữ cảnh). Vấn đề này được giải quyết bằng cách thêm các bước reset context vào harness. Nhưng khi dùng chính harness đó trên Claude Opus 4.5, hành vi này đã biến mất — các bước reset trở thành trọng lượng chết (dead weight). Điều này cho thấy harness cần tiếp tục tiến hóa cùng model.

Managed Agents: một dịch vụ được host trên Claude Platform, chạy các agent chạy dài hạn (long-horizon) thay mặt người dùng thông qua một tập nhỏ các interface được thiết kế để tồn tại lâu hơn bất kỳ triển khai cụ thể nào.

Về xác thực (auth): xác thực có thể được gộp cùng một resource hoặc giữ trong một vault bên ngoài sandbox. Với Git, mỗi repository dùng access token riêng để clone repo trong lúc khởi tạo sandbox, gắn vào git remote cục bộ — push/pull hoạt động từ bên trong sandbox mà agent không bao giờ tự tay xử lý token. Với tool tùy chỉnh, hệ thống hỗ trợ MCP và lưu OAuth token trong vault bảo mật; Claude gọi tool MCP qua một proxy chuyên dụng — proxy này nhận vào một token gắn với phiên (session), lấy thông tin xác thực tương ứng từ vault, và thực hiện lệnh gọi tới dịch vụ bên ngoài. Harness không bao giờ được biết về bất kỳ thông tin xác thực nào.

Về quản lý context dài hạn: tác vụ chạy dài thường vượt quá độ dài context window của Claude, và các cách thông thường để xử lý điều này đều liên quan đến các quyết định không thể đảo ngược về việc giữ lại gì. Trong Managed Agents, phiên (session) cung cấp cùng lợi ích này, đóng vai trò một đối tượng ngữ cảnh sống ngoài context window của Claude. Nhưng thay vì được lưu trong sandbox hay REPL, ngữ cảnh được lưu bền vững trong session log. Interface `getEvents()` cho phép “brain” (phần model) truy vấn ngữ cảnh bằng cách chọn các lát cắt vị trí (positional slices) trong luồng sự kiện — có thể linh hoạt tiếp tục từ nơi dừng đọc lần trước, tua ngược vài sự kiện trước một thời điểm cụ thể để xem bối cảnh dẫn tới nó, hoặc đọc lại ngữ cảnh trước một hành động cụ thể. Bất kỳ sự kiện nào được lấy về cũng có thể được biến đổi trong harness trước khi đưa vào context window của Claude — các phép biến đổi này có thể là bất cứ gì harness mã hóa, bao gồm cả tổ chức lại ngữ cảnh để đạt tỷ lệ cache hit cao và context engineering nói chung.

6.2. “Lessons from Anthropic on building effective human-agent teams”

Nguồn: <https://claude.com/blog/building-effective-human-agent-teams>

Cách làm việc với AI đang chuyển từ trải nghiệm đơn người chơi sang trải nghiệm đa người chơi, nơi con người và agent làm việc cùng nhau như một đội để đạt mục tiêu chung. Một số bài học thực tế từ các đội ngũ kỹ thuật tại Anthropic:

- **Roster (danh sách vai trò):** một đội kỹ thuật tại Anthropic bắt đầu tạo các “roster” để mã hóa vai trò của con người và agent, vì điều này giúp thúc đẩy công việc dễ dàng và cụ thể hơn nhiều.
- **Vai trò cụ thể** giúp con người dễ theo dõi trách nhiệm nằm ở đâu — cho từng tác vụ riêng lẻ hay cho toàn bộ tập trách nhiệm của một đội.

- **Viết file skill để định nghĩa vai trò cụ thể của từng agent** giúp việc chuyên biệt hóa trở nên dễ dàng, và cho phép mọi người trong công ty nhanh chóng dựng lên các agent khác cùng loại.
- **Đội ngũ thêm agent mới** khi tập trung vào các lĩnh vực mới khi dự án phức tạp hơn — ví dụ họ thêm một “release manager agent” để xử lý các bản phát hành phần mềm mới. Những phương pháp này cho phép mô hình tư duy của con người về đội human-agent mở rộng quy mô cùng với số lượng agent tăng lên.
- Dù một số agent chỉ đơn giản hoàn thành các tác vụ được giao, những agent quan trọng nhất còn **chủ động đề xuất các dự án và luồng công việc mới**.
- **Trao quyền tự trị tỉ lệ thuận với độ tin cậy đã chứng minh**, rồi mở rộng dần một cách có chủ đích. Kỹ sư đã điều phối agent xử lý độc lập tới 500 lần sửa lỗi (bug fix), nhưng chắc chắn không bắt đầu như vậy ngay từ đầu. Khi một đồng nghiệp con người mới gia nhập đội, cần thời gian để đánh giá năng lực và phát triển thói quen làm việc vững chắc — điều tương tự cũng đúng với agent. Người dùng phải thử nghiệm giao cho agent nhiều tác vụ khác nhau để học được agent có khả năng gì, cách mô tả mục tiêu rõ ràng, cần file skill nào, và prompt nào hiệu quả để gọi ra hành vi mong muốn. Cũng quan trọng không kém là việc kiểm thử lại các tác vụ khi model thay đổi và cải thiện — prompt có thể cần viết lại, và các rào chắn (guardrails) từng hữu ích có thể lại cản trở một model thông minh hơn theo đuổi giải pháp sáng tạo hơn.
- **Cách xây dựng lòng tin với agent theo thời gian** tại Anthropic: xem xét thủ công công việc của agent ở giai đoạn đầu để kiểm định chất lượng, đưa phản hồi, và thiết kế checklist xác minh tác vụ; yêu cầu agent dùng một “verifier agent” (agent kiểm tra) để kiểm tra công việc của nó như một phần của tác vụ; xây dựng phản chiếu (reflection) vào chu trình và yêu cầu agent tự xem lại các sai sót của mình để công việc cải thiện theo thời gian; theo dõi loại tác vụ nào mỗi agent đã đạt được quyền tự trị và mở rộng phạm vi theo từng loại tác vụ sau các lần thành công lặp lại.
- Ví dụ thực tế: một lãnh đạo kỹ thuật tại Anthropic tiếp quản một đội mới với backlog lớn. Để xử lý, ông mời một vài con người và một vài agent cùng giúp sắp xếp và ưu tiên các việc quan trọng nhất. Một nhóm agent trong đội đọc qua toàn bộ mục trong backlog, xác định xem có ai đang xử lý mục đó không, và gán điểm độ phức tạp cho bất kỳ mục nào chưa có chủ.

7. Tổng hợp nguyên tắc thiết kế then chốt

Bảng dưới đây tổng hợp các nguyên tắc quan trọng nhất rút ra từ toàn bộ tài liệu, được nhóm theo tầng kiến trúc:

7.1. Tầng quyết định có nên xây agent hay không

- Luôn tìm giải pháp đơn giản nhất trước; agent không phải mặc định cho mọi bài toán.
- Hệ agentic đánh đổi độ trễ và chi phí để lấy hiệu năng — cần cân nhắc khi nào đánh đổi này hợp lý.
- Workflow (đường code định sẵn) phù hợp cho tác vụ được định nghĩa rõ ràng, cần tính dự đoán được; agent phù hợp khi cần linh hoạt và ra quyết định do model dẫn dắt.

7.2. Tầng orchestrator/multi-agent

- Orchestrator-worker: một lead agent điều phối, các subagent chuyên biệt chạy song song, mỗi subagent có context window riêng.
- Mỗi subagent cần hợp đồng 4 phần: mục tiêu, định dạng output, hướng dẫn tool/nguồn, ranh giới nhiệm vụ.
- Quy tắc effort-scaling: 1 agent cho tra cứu đơn giản, 2-4 subagent cho so sánh, >10 subagent cho nghiên cứu phức tạp.
- Multi-agent chỉ hiệu quả khi sub-task độc lập thật sự — không phù hợp cho tác vụ có nhiều phụ thuộc chéo (như coding).
- Chi phí: multi-agent tốn ~15 lần token so với chat thường — cần giá trị đủ lớn để bù chi phí.
- Isolation boundary: mặc định cho subagent biết càng ít về nhau càng tốt, để đảm bảo song song thực sự.
- Lưu kế hoạch ra ngoài (externalize) trước khi context đầy, thay vì trông chờ context window lớn hơn.
- Dùng artifact pattern: subagent ghi kết quả chi tiết vào nơi lưu trữ chung, trả về tham chiếu nhẹ thay vì toàn bộ nội dung.

7.3. Tầng context engineering

- Context là tài nguyên hữu hạn với lợi nhuận biên giảm dần — kỷ luật cốt lõi là tìm tập token tín hiệu cao nhỏ nhất.
- System prompt cần ở “độ cao” vừa phải — không hard-code cứng nhắc, không mơ hồ quá mức.
- Context rot là thực tế kiến trúc — càng nhiều token, khả năng nhớ chính xác càng giảm, ngay cả trước khi chạm giới hạn cứng.
- Ba nguyên thủy quản lý context: compaction, tool-result clearing, memory.
- Bộ nhớ bền vững chỉ nên chứa thông tin tiếp tục ràng buộc suy luận tương lai — không lưu tràn lan.
- Skill dùng progressive disclosure: chỉ nạp tên/mô tả khi khởi động, nạp chi tiết khi kích hoạt — cho phép skill “vô hạn” dung lượng.
- Tool: giữ 3-5 tool dùng nhiều nhất luôn nạp sẵn; dùng namespacing khi có nhiều tool; gộp tool nhỏ lẻ thành tool cấp cao có mục đích rõ.

7.4. Tầng loop/planning

- Loop engineering tối ưu cả hệ thống quanh nhiều lượt tương tác: trigger, context, tool, cách xác minh, nơi lưu state, khi nào dừng.
- Verifier là nút thắt cổ chai của bất kỳ vòng lặp nào, không phải bản thân model.
- Cần điểm kiểm tra của con người (human checkpoint) trước các hành động quan trọng/không thể đảo ngược.
- Trao quyền tự trị tỉ lệ thuận với độ tin cậy đã chứng minh, mở rộng dần theo từng loại tác vụ.

8. Áp dụng cụ thể vào bài toán trợ lý pháp lý đa lĩnh vực

1. **Orchestrator nhỏ (kiểu flash-lite):** đúng hướng theo nguyên tắc effort-scaling — orchestrator chỉ cần định tuyến và quyết định số lượng sub-agent cần gọi, không cần model mạnh để làm việc đó. Đây khớp trực tiếp với khuyến nghị “model nhẹ, nhanh cho các tác vụ định tuyến/phân loại đơn giản, model mạnh chỉ dành cho phần việc thực sự đòi hỏi suy luận sâu”.
2. **Mỗi domain-agent (đất đai, thuế, v.v.) = một hợp đồng 4 phần rõ ràng:** mục tiêu, định dạng output, tool/nguồn dữ liệu được phép dùng, và ranh giới nhiệm vụ tường minh (ví dụ: agent thuế không tự trả lời câu hỏi đất đai dù có liên quan mờ nhạt — tránh tình trạng trùng lặp công việc đã ghi nhận ở case ví dụ chip ô tô 2021 vs. chuỗi cung ứng 2025).
3. **Skill gắn theo loại vấn đề pháp lý cụ thể:** áp dụng đúng progressive disclosure — SKILL.md ngắn gọn mô tả “khi nào dùng skill này” (ví dụ: “dùng khi câu hỏi liên quan đến chuyển nhượng quyền sử dụng đất”), logic chi tiết (cách truy vấn GraphRAG/Neo4j, cách gọi API thuvienphapluat.vn, cách xử lý các trường hợp ngoại lệ trong luật) để trong file riêng, chỉ nạp khi skill được kích hoạt.
4. **Context/memory:** bắt đầu đơn giản với “ghi ra scratchpad + compaction” trước khi xây memory phức tạp; giữ orchestrator gọn nhẹ — đẩy trạng thái chi tiết (lịch sử tra cứu, ngữ cảnh phiên làm việc) xuống từng domain-agent theo nguyên tắc isolate.
5. **Planning/checklist:** đây là nơi loop engineering áp dụng trực tiếp — orchestrator (hoặc một “planner agent” tách riêng) tạo checklist các bước, giao việc cho domain-agent, có một verifier riêng kiểm tra kết quả (độ chính xác pháp lý, trích dẫn nguồn đúng điều khoản) trước khi xuất báo cáo PDF hoặc gửi email qua hành động MCP.
6. **Thứ tự triển khai đề xuất, theo đúng nguyên tắc “bắt đầu đơn giản, đo lường, rồi mới thêm phức tạp”:**
 - **Bước 1:** xây 1 domain-agent đơn (ví dụ agent luật thuế) hoạt động độc lập, đo lường chất lượng trả lời trong đúng phạm vi của nó.
 - **Bước 2:** thêm orchestrator + 2–3 domain-agent khi thấy giá trị vượt chi phí token tăng thêm (nhớ hệ số tham khảo ~15 lần so với chat đơn).
 - **Bước 3:** thêm tầng context engineering (memory, compaction, skill) khi agent bắt đầu cần xử lý phiên làm việc dài hoặc nhiều lượt hỏi liên quan tới nhau.
 - **Bước 4:** thêm tầng loop/verifier khi cần agent tự động hoàn toàn một quy trình nhiều bước (nghiên cứu một bộ luật → tổng hợp → xuất PDF → gửi email), với các human checkpoint ở những điểm quan trọng (ví dụ trước khi gửi email hoặc công bố kết luận pháp lý).
7. **Tận dụng hạ tầng đã có:** hệ thống GraphRAG luật Việt Nam (Neo4j) đang xây có thể trở thành chính **tool retrieval** cho domain-agent luật đất đai/luật thuế — không cần xây lại hệ tri thức riêng cho từng agent, chỉ cần định nghĩa rõ tool truy vấn đồ thị tri thức (ví dụ `search_land_law`, `search_tax_law`) theo namespacing đã mô tả ở mục 4.5.

9. Danh mục toàn bộ nguồn tham khảo

Giai đoạn 1 — Agent cơ bản 1. Building Effective Agents (Anthropic) — <https://www.anthropic.com/research/effective-agents> 2. Reference implementation, claude-cookbooks — <https://github.com/anthropics/claude-cookbooks/tree/main/patterns/agents>

Giai đoạn 2 — Multi-agent 3. How we built our multi-agent research system (Anthropic Engineering) — <https://www.anthropic.com/engineering/multi-agent-research-system> 4. Anthropic's Multi-Agent Blueprint: What Production Adds — <https://fountaincity.tech/resources/blog/anthropic-multi-agent-blueprint-production/> 5. Building Effective AI Agents: Architecture Patterns and Implementation Frameworks (whitepaper) — <https://resources.anthropic.com/building-effective-ai-agents>

Giai đoạn 3 — Context Engineering 6. Effective context engineering for AI agents (Anthropic Engineering) — <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents> 7. Context engineering: memory, compaction, and tool clearing (Claude Cookbook) — <https://platform.claude.com/cookbook/tool-use-context-engineering-context-engineering-tools> 8. Equipping agents for the real world with Agent Skills (Anthropic Engineering) — <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills> 9. Context Engineering (LangChain) — <https://www.langchain.com/blog/context-engineering-for-agents> 10. Writing effective tools for AI agents — using AI agents (Anthropic Engineering) — <https://www.anthropic.com/engineering/writing-tools-for-agents>

Giai đoạn 4 — Loop Engineering 11. Loop Engineering: The 2026 Guide to AI Agent Loops — <https://www.jacknjoroge.com/loop-engineering/> 12. What Is Loop Engineering? Beyond Prompt Engineering — <https://explainx.ai/blog/what-is-loop-engineering-ai-agents-2026> 13. What Is Loop Engineering? A Complete Guide from Prompt to Harness Engineering (Tosea.ai) — <https://tosea.ai/blog/loop-engineering-ai-agents-complete-guide-2026> 14. Loop Engineering: A Guide for Engineers and Practitioners (Medium, Adnan Masood) — <https://medium.com/@adnanmasood/loop-engineering-a-guide-for-engineers-and-practitioners-893bb65ea943> 15. Loop Engineering: The Guide for AI Agents (Lushbinary) — <https://lushbinary.com/blog/loop-engineering-ai-coding-agents-guide/> 16. What Is Loop Engineering? The New Meta for AI Coding Agents (MindStudio) — <https://www.mindstudio.ai/blog/what-is-loop-engineering-ai-coding-agents> 17. Loop Engineering Guide 2026 (AI Builder Club) — <https://www.aibuilderclub.com/blog/loop-engineering-guide-2026> 18. Loop Engineering: How to Design Coding Agent Loops (explainx.ai) — <https://explainx.ai/blog/loop-engineering-coding-agents-claude-code-guide-2026>

Tài liệu bổ trợ 19. Scaling Managed Agents: Decoupling the brain from the hands (Anthropic Engineering) — <https://www.anthropic.com/engineering/managed-agents> 20. Lessons from Anthropic on building effective human-agent teams — <https://claude.com/blog/building-effective-human-agent-teams>

Tài liệu này được tổng hợp và diễn giải lại bằng lời văn riêng từ các nguồn công khai nêu trên, phục vụ mục đích tham khảo cá nhân. Để có nội dung đầy đủ, chính xác nhất, vui lòng truy cập trực tiếp các đường link nguồn.